

Relatório Trabalho Final
INF05027 - Projeto e Análise de Algoritmos I
Turma B
Trabalho Final

André Schaidhauer Luckmann
Cartão UFRGS: 00601117

Junho de 2026

Sumário

1	Introdução	1
2	Problema 1: O Rato no Labirinto	2
2.1	Descrição do Problema	2
2.2	Estratégia da Solução	2
2.3	Argumento de Correção	3
2.4	Análise de Complexidade	3
2.5	Discussão	4
3	Problema 2: Ir e Vir	4
3.1	Descrição do Problema	4
3.2	Estratégia da Solução	4
3.3	Argumento de Correção	5
3.4	Análise de Complexidade	6
3.5	Discussão	6
4	Conclusão	6

1 Introdução

Este relatório descreve as soluções desenvolvidas para o trabalho final da disciplina de **Projeto e Análise de Algoritmos I**, cujo objetivo é consolidar os estudos de algoritmos sobre grafos por meio da implementação e análise de soluções para dois problemas na plataforma Beecrowd:

- **Problema 1:** *O Rato no Labirinto* (<https://judge.beecrowd.com/pt/problems/view/1799>).
- **Problema 2:** *Ir e Vir* (<https://judge.beecrowd.com/pt/problems/view/1128>).

A linguagem escolhida para ambas as implementações foi **Python** por já ser utilizada como linguagem principal da disciplina, possuindo vários templates de código que estudamos e por ser uma linguagem intuitiva que oferece bibliotecas com diversas estruturas de dados completas. Além disso, o autor (André) já possuía experiência e familiaridade com **Python**.

2 Problema 1: O Rato no Labirinto

2.1 Descrição do Problema

O ratinho IBO deve atravessar um labirinto representado como um grafo não-direcionado. Ele parte de um vértice rotulado **Entrada**, deve obrigatoriamente passar pelo vértice que contém o **queijo**, identificado pelo caractere *****, e terminar no vértice **Saida**. IBO deve seguir sempre o caminho mais curto em seus trajetos. Caso seja necessário passar por um mesmo ponto duas vezes (uma indo até o queijo, outra voltando até a saída), *cada visita é contada separadamente*.

Dada a descrição do labirinto na entrada (número de pontos, número de ligações e a lista de pares de vértices ligados), o objetivo é imprimir o número mínimo de pontos pelos quais IBO passa para cumprir a tarefa.

2.2 Estratégia da Solução

Modelagem. O labirinto é representado como um grafo não-direcionado $G = (V, E)$, em que V é o conjunto de pontos rotulados do labirinto (palavras compostas apenas por letras, como A, F, **Entrada**, **Saida**, e o queijo *****) e E é o conjunto de ligações. As arestas do grafo não são valoradas, pois não é fornecida essa informação e estamos interessados na quantidade de pontos visitados (tamanho do caminho).

Decomposição. A observação central é que o caminho ótimo $\text{Entrada} \rightarrow \dots \rightarrow * \rightarrow \dots \rightarrow \text{Saida}$ pode ser decomposto em dois caminhos mínimos *independentes*:

- (a) caminho mínimo de **Entrada** até *****;
- (b) caminho mínimo de ***** até **Saida**.

Como o problema explicita que cada visita conta separadamente, não há ganho em tentar reaproveitar trechos comuns: a soma dos comprimentos dos dois caminhos mínimos é justamente o número total mínimo de visitas, conforme demonstrado na Seção 2.3.

Algoritmo. Aplica-se Busca em Largura (BFS) duas vezes:

1. `bfs_distance(graph, start_node="Entrada", target_node="*")`
2. `bfs_distance(graph, start_node="*", target_node="Saida")`

e a resposta é a soma dos dois valores retornados.

Estruturas de dados.

- `graph`: `Dict[Node, List[Node]]` - grafo representado como lista de adjacências usando um dicionário.
- `collections.deque` - Double-ended queue, utilizado somente como fila de vértices a serem visitados.
- `visited`: `Dict[Node, bool]` - controle de vértices já visitados.
- `distances`: `Dict[Node, int]` - distância (em número de arestas) de cada vértice ao vértice de partida da busca em largura.

Justificativa de design. A BFS é *ótima* para encontrar caminhos mínimos em grafos não valorados, com custo linear no tamanho do grafo. O algoritmo de Dijkstra foi considerado inicialmente, mas teria um custo adicional desnecessário (heap). Como a busca em largura vai passando por "níveis" do grafo, ela é uma forma mais otimizada que a DFS caso o algoritmo de DFS entre em uma ramificação muito grande do grafo que não leve ao nodo desejado (pior caso).

2.3 Argumento de Correção

Lema 1 (Corretude da BFS em grafos não valorados). *A BFS, partindo de um vértice s , atribui a cada vértice u alcançável exatamente a distância em arestas $d(s, u)$.*

Esboço. Por indução sobre as camadas de descoberta. A camada 0 é $\{s\}$, com $d = 0$. Suponha que ao final do processamento da camada k todos os vértices alcançáveis a distância exatamente k tenham sido marcados e enfileirados com distância correta. Quando um vértice v da camada k é desenfileirado, examina-se cada vizinho $w \in N(v)$: se w ainda não está marcado, ele é colocado na fila com $\text{distances}[w] = \text{distances}[v] + 1 = k + 1$. Como a fila obedece a lógica FIFO (First-In-First-Out) e somente vértices das camadas anteriores foram enfileirados antes, nenhum vértice cuja distância seja menor que $k + 1$ poderia ainda estar não marcado. Logo, todos os vértices à distância $k + 1$ são descobertos com a distância correta. Pela indução, a BFS é correta. $\square$

Lema 2 (Decomposição). *Seja Opt o número mínimo de pontos visitados em qualquer caminho $\text{Entrada} \rightarrow \dots \rightarrow * \rightarrow \dots \rightarrow \text{Saida}$ em G . Então*

$$\text{Opt} = d(\text{Entrada}, *) + d(*, \text{Saida}),$$

onde $d(\cdot, \cdot)$ é a distância em arestas em G .

Esboço. ($\leq$) Sejam P_1 um caminho mínimo de Entrada a $*$ (com $d(\text{Entrada}, *)$ arestas, ou seja, $d(\text{Entrada}, *) + 1$ vértices contados sem repetição), e P_2 um caminho mínimo análogo de $*$ a Saida . A concatenação $P_1 \cdot P_2$ é um caminho válido para o problema, e como o enunciado conta a passagem por $*$ duas vezes (uma vez ao final de P_1 e outra ao início de P_2), o número total de visitas é exatamente $|P_1| + |P_2| = d(\text{Entrada}, *) + d(*, \text{Saida})$. Logo, $\text{Opt} \leq d(\text{Entrada}, *) + d(*, \text{Saida})$.

($\geq$) Reciprocamente, qualquer caminho válido P pode ser particionado pela primeira ocorrência de $*$ em sub-caminhos P'_1 (de Entrada a $*$) e P'_2 (de $*$ a Saida). Cada um deles é um caminho em G , portanto $|P'_1| \geq d(\text{Entrada}, *)$ e $|P'_2| \geq d(*, \text{Saida})$. Somando, o número total de visitas em P é $\geq d(\text{Entrada}, *) + d(*, \text{Saida})$. Em particular, $\text{Opt} \geq d(\text{Entrada}, *) + d(*, \text{Saida})$.

Combinando as duas desigualdades, vale a igualdade. $\square$

Validade da implementação. As duas chamadas de `bfs_distance` terminam, pois o grafo é finito e cada vértice é enfileirado no máximo uma vez (após a marcação). O resultado é sempre não-negativo. Caso o vértice-alvo seja inalcançável, a função retorna -1 ; o enunciado garante a existência dos caminhos, de modo que esse caso não ocorre nas instâncias do problema.

2.4 Análise de Complexidade

Dados os limites de Pontos ($4 \leq \text{Pontos} \leq 4000$) e Ligacoes ($4 \leq \text{Ligacoes} \leq 5000$), sejam $n = |V| \leq 4000$ e $m = |E| \leq 5000$. Analisa-se função a função:

- `build_graph_from_input`: lê m linhas. Cada inserção em `dict` e em `list` é $O(1)$ amortizado. **Tempo:** $O(m)$. **Memória:** $O(n + m)$ (lista de adjacências).
- `bfs_distance`: a inicialização de `visited` e `distances` percorre todos os vértices, custando $O(n)$. O laço principal enfileira cada vértice no máximo uma vez ($O(n)$) e percorre cada aresta no máximo duas vezes (uma por extremidade), custando $O(m)$. **Tempo total:** $O(n + m)$. **Memória adicional:** $O(n)$.

- **main:** realiza duas chamadas independentes de BFS. Custo total: $2 \cdot O(n+m) = O(n+m)$.

Resumo. **Complexidade de tempo:** $O(n+m)$. **Complexidade de memória:** $O(n+m)$, dominada pela lista de adjacências.

2.5 Discussão

- **Dificuldades encontradas.** O ponto não-trivial foi interpretar corretamente a regra de contagem de visitas: como cada passagem por um ponto conta como uma visita independente, a decomposição em duas BFS independentes torna-se ótima e simples. Sem essa observação, é tentador buscar uma solução mais elaborada que tente compartilhar trechos.
- **Alternativas consideradas.**
 - *Dijkstra*: aplicável, mas com sobrecarga de fila de prioridade desnecessária para arestas unitárias.
 - *DFS*: não garante caminho mínimo, portanto inadequada.
- **Possíveis otimizações.**
 - Encerrar a BFS no instante em que o vértice-alvo é *desenfileirado*, em vez de detectá-lo na inserção, é uma refatoração equivalente em corretude e talvez mais idiomática.
- **Aprendizados.** A BFS pode ser vista como caso particular de Dijkstra quando todas as arestas têm o mesmo peso.

3 Problema 2: Ir e Vir

3.1 Descrição do Problema

Em uma certa cidade, há N interseções ligadas por ruas que podem ser de mão única ou mão dupla. Deseja-se verificar a propriedade de *conexidade*: dadas quaisquer duas interseções V e W , é possível viajar de V para W e de W para V .

A entrada contém vários casos de teste que fornecem N , M , seguidos de M linhas descrevendo M ruas no formato (V, W, P) onde $P = 1$ indica via de mão única $V \rightarrow W$ e $P = 2$ indica via de mão dupla $V \leftrightarrow W$. A sequência termina com uma linha contendo $N=0$ e $M=0$. Para cada caso, deve-se imprimir 1 se a propriedade é satisfeita ou 0 caso contrário.

3.2 Estratégia da Solução

Modelagem. Cada caso de entrada é modelado como um dígrafo $G = (V, A)$. Uma rua de mão única $V \rightarrow W$ contribui com a aresta (v, w) e uma rua de mão dupla $V \leftrightarrow W$ contribui com as duas arestas (v, w) e (w, v) . A propriedade do enunciado “ir de V a W e voltar de W a V para todo par” é uma característica de um componente fortemente conexo em um grafo. Desta forma, para resolver o problema, a interpretação foi de que deve-se verificar se G é composto por um único componente fortemente conexo (SCC, Strongly-Connected-Component) que contém todos os vértices.

Algoritmo. Aplica-se o algoritmo **Kosaraju-Sharir** para encontrar os SCCs, com uma otimização: basta saber se há *exatamente um* SCC. Portanto, ao detectar a necessidade de iniciar uma segunda busca DFS, o algoritmo retorna 0 imediatamente.

Etapas.

1. Construir o grafo invertido G^T (`reverse_graph`).
2. Executar uma DFS em G^T produzindo a ordem reversa de finalização (ordenação topológica via DFS recursivo, função `dfs_toposort`).
3. Executar uma DFS em G visitando os vértices na ordem obtida na etapa anterior pela função `dfs_toposort`. Cada chamada de DFS a partir de um vértice ainda não visitado descobre exatamente um SCC.
4. Contar quantas chamadas de DFS são necessárias para cobrir todos os vértices: se for mais que uma, retornar 0; caso contrário, retornar 1.

Estruturas de dados.

- `graph`: `Dict[Node, List[Node]]` - grafo representado como lista de adjacências usando um dicionário.
- `rev_g`: `Dict[Node, List[Node]]` - grafo inverso representado como lista de adjacências usando um dicionário.
- `visited`: `Dict[Node, bool]` - controle de vértices já visitados.
- `order`: `list[Node]` - lista com a ordem inversa de vértices visitados por DFS em G^T .

3.3 Argumento de Correção

Teorema 1 (Kosaraju-Sharir). *Os componentes fortemente conexos de um dígrafo G são as árvores da floresta DFS executada em G visitando-se os vértices na ordem inversa de visita da DFS feita em G^T .*

Esboço. Seja $\text{SCC}(G)$ o conjunto de SCCs de G e considere o *grafo de componentes* $\mathcal{C}(G)$, que é acíclico (DAG). A primeira DFS em G^T visita os nodos em uma ordem determinada. A propriedade chave é: o último vértice a ser visitado na DFS em G^T pertence a um SCC que é *fonte* em $\mathcal{C}(G^T)$, e ao mesmo tempo é *sumidouro* em $\mathcal{C}(G)$. Iniciando uma DFS em G a partir desse vértice, descobrem-se exatamente os vértices do seu SCC, pois *nenhuma aresta sai* desse SCC em G para outros SCCs (por ele ser sumidouro em $\mathcal{C}(G)$). Por indução, removendo o SCC já descoberto, o argumento se repete para o próximo vértice de maior tempo de finalização ainda não visitado. Logo, cada chamada de DFS na fase final descobre exatamente um SCC. $\square$

Adaptação à implementação. O código aplica a ordenação topológica por DFS no grafo *invertido* (`rev_g`) e a DFS final no grafo *original* (`graph`). Como $(G^T)^T = G$, a ordem produzida é equivalente à ordem inversa de visita exigida pelo Teorema 1. Em particular, cada execução de `dfs(graph, root)` dentro do laço final descobre um único SCC de G .

Critério de aceitação. O dígrafo é fortemente conexo se, e somente se, $|\text{SCC}(G)| = 1$, isto é, uma única chamada de DFS na fase final cobre todos os vértices. Na função `is_connected`, o contador `connected_components` é incrementado a cada nova raiz de DFS; quando uma segunda raiz seria iniciada, a função retorna 0.

Validade. O grafo é finito e ambas as DFS terminam. A saída é sempre binária. O *early-exit* não compromete a corretude, pois bastam dois SCCs distintos para que a resposta seja 0.

3.4 Análise de Complexidade

Sejam $n = |V| \leq 2000$ e $m = |A| \leq n(n-1)/2$. Por caso de teste:

- **build_graph_from_input:** São lidas m linhas, logo a complexidade de tempo é $O(m)$. Como o grafo é armazenado em memória como uma lista de adjacência usando um dicionário, o custo de memória é $O(n+m)$.
- **reverse_graph:** percorre cada par de (nodo, vizinho) uma vez ao construir as adjacências invertidas. $O(n+m)$ tempo e $O(n+m)$ memória.
- **dfs_toposort:** a DFS clássica visita cada vértice uma vez (custo $O(n)$) e cada aresta uma vez (custo $O(m)$); a inversão de **order** ao final é $O(n)$. Complexidade de tempo total é $O(n+m)$. O dicionário **visited** requer memória adicional com custo $O(n)$.
- **is_connected:** além das chamadas acima, executa no máximo uma DFS completa adicional. Pior caso $O(n+m)$.

Resumo por caso de teste: **Tempo:** $O(n+m)$, **Memória:** $O(n+m)$.

Custo total. Para t casos de teste, o custo total é $O(t \cdot (n+m))$.

3.5 Discussão

- **Decisões.** A percepção mais importante foi reconhecer que a propriedade “viajar de V a W e de W a V para todo par” é uma característica de um componente fortemente conexo em um dígrafo. A partir disso, consultando os materiais de aula e [repositório no Github](#), foi encontrado o algoritmo de Kosaraju-Sharir que encontra os SCCs de um dígrafo. Dessa forma, analisando o problema foi constatado que o programa deveria verificar se o dígrafo é um SCC que contém todos os nodos do dígrafo.
- **Alternativas consideradas.** Inicialmente, havia pensado em realizar buscas DFS/BFS entre um nodo fonte e um nodo alvo, para todos os nodos do grafo, e caso não fosse possível satisfazer algum caminho, o programa deveria retornar 0. Porém, essa alternativa provavelmente seria força bruta e ineficiente, resultando em uma complexidade de ordem cúbica $O(n^3)$.
- **Possíveis otimizações.**
 - Como o algoritmo apenas precisa saber se há um único componente fortemente conexo, o contador **connected_components** poderia ser substituído por uma simples **flag** booleana.
- **Aprendizados.** O algoritmo de Kosaraju-Sharir mostra como a ordem de visitas da DFS no grafo transposto induz uma ordem topológica do DAG de componentes. Também demonstra a propriedade dos sumidouros do grafo de componentes: o SCC do último vértice visitado em G^T é um sumidouro em $\mathcal{C}(G)$, do qual nenhuma aresta sai para outros SCCs.

4 Conclusão

Este trabalho consolidou conceitos centrais de algoritmos sobre grafos por meio da implementação de duas soluções para problemas de Grafos:

- Para o problema **O Rato no Labirinto**, foi mostrado que o problema de caminho mínimo passando obrigatoriamente por um vértice intermediário pode ser decomposto em duas BFSs independentes em um grafo não valorado, resultando em uma solução linear no tamanho do grafo, com custo $O(n + m)$.
- Para o problema **Ir e Vir**, foi aplicado o algoritmo de Kosaraju-Sharir para verificar a quantidade de componentes fortemente conexos do dígrafo, retornando 0 em caso de detecção de mais de um componente fortemente conexo no dígrafo. A solução possui complexidade $O(n + m)$ por caso de teste.

Ambas as soluções respeitam os limites de entrada do juiz e foram aceitas pelo Beecrowd. Além do desenvolvimento de código correto, o exercício reforçou a importância de modelar adequadamente o problema (identificar se o grafo é direcionado ou não, peso das arestas, decomposição em sub-problemas) e de fornecer argumentos de correção formais e análise de complexidade.